

Chat transcript

User

commonalities and differences of the recommended directory structures of analog/mixed signal/digital Makefiles in <https://github.com/iic-jku/ihp-sg13cmos5l-ams-chip-template> and https://github.com/HeiChips/heichips26-template/tree/main/macros/heichips26_analog_project Look in [[https://heichips.github.io/heichips26-analog-workshop/%5D\(https://heichips.github.io/heichips26-analog-workshop/\)](https://heichips.github.io/heichips26-analog-workshop/%5D(https://heichips.github.io/heichips26-analog-workshop/)) and <https://github.com/iic-jku/analog-circuit-design> for additional documentation

Assistant

I'll compare the two templates at the level of actual directory layout and Makefile responsibility, then use the workshop/docs repo to explain which differences are intentional workflow differences versus incidental repository organization.

Assistant

One important distinction is already clear: the JKU repository is a **whole AMS-chip hierarchy** (chip-level Makefile + per-macro Makefiles), whereas `heichips26_analog_project` is a **submission-sized analog-on-top project** that recursively contains an analog sub-macro. I'm checking the actual Makefiles now, because the directory names look similar but their contracts differ.

Assistant

The strongest commonality is not just "similar folders": the HeiChips analog Makefiles are clearly derived from the same JKU/IIC-OSIC flow conventions—TOP, CELL, schematic/xschem, testbenches/xschem, layout, netlist/{schematic,layout,pex}, verification/{drc,lvs}, final/*, and make all as an orchestration target. I'm separating genuine structural deltas from mere naming/version drift.

Assistant

Yes. I read "directory structures ... of the Makefiles" as **the directory contract assumed/recommended by the Makefiles**, rather than merely where the Makefiles themselves happen to reside.

The short version is: **the two repositories now describe essentially the same design-flow architecture at macro level**. The major differences are not competing philosophies but **different hierarchy levels and use cases**:

- `ihp-sg13cmos5l-ams-chip-template` is a **complete AMS chip hierarchy**.
- `heichips26-template` gives the user **one submission macro**, either analog-on-top or digital-on-top, which may recursively contain smaller macros.
- The **analog leaf-macro structure is almost the same** in both.

- The **digital leaf-macro structure is likewise very similar**, but naturally adds RTL, LibreLane, cocotb/Verilog, FPGA, synthesized netlists, etc.
- The JKU full-chip top level is the actual **mixed-signal superset** of the analog and digital structures. (github.com)

1. The common mental model

The basic abstraction in both projects is:

```
design-unit/
├── Makefile
├── README.md
├── source design
│   ├── schematic/...
│   ├── rtl/...           # if digital
│   └── layout/...        # if manually laid out
├── testbenches/...
├── flow/...              # automated implementation, mainly digital
├── scripts/...
├── netlist/...           # generated/intermediate representations
├── verification/...      # reports
├── final/...             # artifacts exported to the parent level
├── macros/...            # optional child design units
│   ├── child/
│   │   ├── Makefile
│   │   └── ...
```

That recursive macros/ idea is explicit in HeiChips: both the analog and digital project can contain sub-macros, each with its own Makefile, and the parent invokes them with \$(MAKE) -C ... all. (github.com)

The JKU AMS template uses exactly the same principle: every component under macros/ and ip/ has its own Makefile/README, can be built independently, and is recursively invoked from the parent. (github.com)

2. Analog leaf macro: almost the same structure

This is probably the most useful comparison.

JKU macros/inverter

The Makefile literally declares:

```
schematic/xschem  
testbenches/xschem  
layout  
scripts  
testbenches/xschem/plot_simulations
```

```
final/gds  
final/lef  
final/lib  
final/vh
```

```
render/img
```

```
netlist/schematic  
netlist/layout  
netlist/pex
```

```
verification/cace  
verification/lvs  
verification/drc
```

([raw.githubusercontent.com](https://raw.githubusercontent.com/githubuser/content.com))

HeiChips heichips26_analog_project/macros/inverter

It declares:

```
schematic/xschem  
testbenches/xschem  
layout  
scripts  
testbenches/xschem/plot_simulations
```

```
final/gds  
final/lef  
final/lib  
final/vh  
final/render
```

```
netlist/schematic  
netlist/layout
```

netlist/pex

verification/cace

verification/lvs

verification/drc

(raw.githubusercontent.com)

So the important conclusion is:

These are not merely similar. They are the same basic analog macro convention.

The most visible directory difference is:

JKU:

render/img/

HeiChips:

final/render/

That's a minor organizational choice. JKU separates documentation/debug rendering from integration deliverables, while the HeiChips project puts the render beside the other final/ artifacts.

Semantic division

Directory	Intended meaning
schematic/xschem/	design source
testbenches/xschem/	simulation stimulus/measurement source
layout/	manually authored physical-design source
scripts/	design/flow automation
netlist/schematic/	generated LVS reference
netlist/layout/	generated extracted LVS representation
netlist/pex/	generated parasitic SPICE
verification/drc/	DRC results
verification/lvs/	LVS results
verification/cace/	characterization specification/results machinery
final/{gds,lef,lib,vh}/	interface to the next hierarchy level

That last distinction is particularly useful: **layout/ is working/source layout; final/gds/ is a published macro artifact.**

The workshop documentation describes the same separation almost verbatim: schematic, testbenches, layout, three kinds of netlists, verification, scripts, and final/ deliverables. (heichips.github.io)

3. One significant analog difference: inverter versus inverter_top

The current JKU example is slightly more sophisticated structurally.

Its analog macro has

TOP = inverter_top

and verifies both

inverter

inverter_top

with *-verify-all. (raw.githubusercontent.com)

The HeiChips workshop/submacro has simply

TOP = inverter

and its *-verify-all verifies just that cell. (raw.githubusercontent.com)

This illustrates an important point about the directory convention:

you do not make one directory per layout cell.

A macro directory may contain several related cells:

```
macros/my_amplifier/  
  schematic/xschem/  
    input_stage.sch  
    output_stage.sch  
    my_amplifier_top.sch  
  layout/  
    input_stage.gds  
    output_stage.gds  
    my_amplifier_top.gds  
  ...
```

and CELL=<name> selects which of them the generic DRC/LVS/PEX machinery operates on.

That is a better mental model than:

```
macros/input_stage/  
macros/output_stage/  
macros/my_amplifier/
```

unless those really are reusable independently published macros.

4. HeiChips analog-on-top is one hierarchy level higher

macros/heichips26_analog_project/ is therefore **not exactly equivalent** to JKU's macros/inverter/.

It is closer to an **analog subsystem/top macro**, containing:

```
heichips26_analog_project/  
├── floorplan/  
├── layout/  
├── schematic/xschem/  
├── testbenches/xschem/  
├── scripts/  
├── netlist/  
├── verification/  
├── final/  
└── macros/  
    ├── inverter/  
    │   ├── Makefile  
    │   └── ...
```

(github.com)

The parent Makefile explicitly does:

```
build-inverter:  
    $(MAKE) -C $(MACROS_DIR)/inverter all
```

```
build-macros:  
    $(MAKE) build-inverter
```

and then:

```
all:  
    $(MAKE) build-macros  
    $(MAKE) klayout-verify-all  
    $(MAKE) magic-verify-all  
    $(MAKE) build-top  
    $(MAKE) sim-all
```

(github.com)

So the hierarchy is deliberately:

```
analog project  
  ↓  
analog submacro(s)
```

not one huge flat analog project.

The HeiChips README explicitly says that flattening everything into the top-level layout is possible but **not recommended**. (github.com)

5. Digital macro: same outer architecture, different contents

Digital uses the same architectural categories but substitutes the digital flow artifacts.

The JKU counter Makefile defines:

```
rtl/

testbenches/
├─ cocotb/
├─ verilog/
└─ xschem/

schematic/xschem/

scripts/
render/img/
verification/

flow/
├─ librelane/
└─ final/

netlist/
├─ nl/
├─ pnl/
├─ spice/
├─ xspice/
└─ pex/

final/
```

with the hardened GDS considered the layout used for physical verification:

```
LAY_DIR := final/gds
```

(raw.githubusercontent.com)

That is an important analog/digital distinction:

Analog

```
layout/*.gds
  |
  └─ DRC/LVS/PEX
    ↓
final/gds/*.gds
```

The designer creates the layout.

Digital

```
rtl/*.sv
  |
  ↓
flow/librelane/
  |
  ↓
final/gds/*.gds
  |
  └─ physical verification / PEX
```

LibreLane creates the layout.

So **layout/ is fundamental for analog but often unnecessary for a digital macro**. In the digital Makefile, `LAY_DIR := final/gds` reflects exactly that distinction. ([raw.githubusercontent.com](https://raw.githubusercontent.com/HeiChips/HeiChips/master/digital/Makefile))

6. HeiChips digital structure follows the same convention

The HeiChips digital project top declares:

```
macros/
rtl/

testbenches/
├─ cocotb/
└─ verilog/

verification/

netlist/
├─ nl/
```

```
|— pnl/  
|— spice/
```

```
flow/  
|— librelane/  
|— final/
```

final/

(raw.githubusercontent.com)

Its all flow is:

```
lint  
→ build child macros  
→ FPGA build  
→ LibreLane top build  
→ simulation
```

(raw.githubusercontent.com)

Its counter submacro is fuller because it is a reusable hardened macro and therefore also contains:

```
schematic/xschem  
testbenches/xschem  
netlist/xspice  
netlist/pex
```

for mixed-signal use and post-layout extraction. (raw.githubusercontent.com)

That again demonstrates a useful rule:

A **digital macro intended to participate in AMS simulation** grows analog-facing representations: Xschem symbol/schematic, XSPICE/SPICE and PEX.

The underlying design remains digital.

7. What is the “mixed-signal directory structure”?

This is where the two repositories differ most substantially.

JKU

The repository root is itself the **AMS chip design unit**:

```

chip_top/
├── rtl/
├── schematic/xschem/
├── testbenches/
│   ├── cocotb/
│   └── xschem/
├── layout/
├── flow/librelane/
├── netlist/
│   ├── schematic/
│   ├── layout/
│   ├── pex/
│   ├── nl/
│   ├── pnl/
│   └── spice/
├── verification/
├── macros/
│   ├── counter/
│   └── inverter/
├── ip/
├── packaging/
├── release/
└── ...

```

The root Makefile consequently defines *both* analog and digital directories:

```

XSCHM_SCH_DIR := schematic/xschem
XSCHM_TB_DIR  := testbenches/xschem
COCOTB_DIR    := testbenches/cocotb
LAY_DIR       := layout
SRC_DIR       := rtl
IP_DIR        := ip
MACROS_DIR    := macros

NET_SCH_DIR    := netlist/schematic
NET_LAY_DIR    := netlist/layout
NET_PEX_DIR    := netlist/pex
NET_PNL_DIR    := netlist/pnl
NET_NL_DIR     := netlist/nl
NET_SPICE_DIR  := netlist/spice

LIBRELANE_DIR := flow/librelane
...

```

(raw.githubusercontent.com)

That is essentially:

mixed-signal = union of analog and digital representations + hierarchy/integration.

HeiChips

There is deliberately **no user-owned whole-chip AMS equivalent** inside heichips26-template.

Instead you choose:

```
macros/  
├─ heichips26_analog_project/  
└─ heichips26_digital_project/
```

and submit one of them. The repository explicitly says these are the two example top-level macros and that the submitted project is subsequently connected to the eFPGA and integrated into the HeiChips chip. (github.com)

Thus:

```
JKU template:  
user controls chip  
├─ analog  
├─ digital  
├─ padframe  
├─ top-level P&R  
└─ packaging
```

```
HeiChips template:  
user controls submission macro
```

↓

```
HeiChips tapeout integration controls chip
```

That explains a lot of apparent structural disagreement.

8. final/ means something especially useful

Both designs converge on the idea that hierarchical macro boundaries should expose **tool-neutral artifacts**.

For analog HeiChips:

```
final/  
├── gds/  
├── lef/  
├── lib/  
├── render/  
└── vh/
```

(github.com)

The JKU analog macro likewise creates GDS, LEF, Liberty and a Verilog stub for top-level integration. (raw.githubusercontent.com)

This gives a very clean interface:

internal implementation

```
schematic/  
layout/  
rtl/  
testbenches/  
verification/  
scripts/  
flow/
```

public macro interface

```
final/  
  gds      physical geometry  
  lef      abstract physical interface  
  lib      timing/interface model  
  vh       HDL black-box interface
```

I would preserve this distinction in any derived project.

9. Source vs generated files

Another convention common to the projects is worth making explicit:

Mostly source-controlled design input

```
rtl/  
schematic/  
layout/  
testbenches/
```

scripts/
flow/
verification/cace/*.yaml
Makefile
README.md

Primarily generated material

testbenches/.../simulations/
testbenches/.../plot_simulations/data/
netlist/
verification/drc/
verification/lvs/
flow/.../runs/

Generated, but intentionally publishable/integratable

final/

The workshop explicitly describes `netlist/` as exported netlists, `verification/` as reports and `final/` as top-level integration deliverables; `make clean` removes those generated products. (heichips.github.io)

That separation is one of the strongest parts of this convention.

10. Where CACE belongs

Both analog macro Makefiles put CACE here:

verification/
└─ cace/

rather than:

testbenches/cace/

(raw.githubusercontent.com)

Conceptually that makes sense because CACE is not merely “run this waveform simulation”; it expresses **specification/characterization over design variation**.

The analog-circuit-design material supports that distinction particularly well: it treats ordinary circuit testbenches separately from PVT/process variation and Monte Carlo/mismatch characterization, and recommends using both corners and Monte Carlo where appropriate. (iic-jku.github.io)

Thus I'd regard:

testbenches/xschem/

as **individual experiments**, and

verification/cace/

as **design-space/specification characterization**.

That's a useful semantic distinction, not merely a historical accident.

11. The differences in one table

Feature	JKU analog leaf	HeiChips analog leaf	HeiChips analog project	Digital leaf
schematic/xschem	yes	yes	yes	usually symbol/AMS view
testbenches/xschem	yes	yes	yes	often yes
layout	source	source	source	normally generated instead
rtl	no	no	no/optional	source
flow/librelane	no	no	no	yes
testbenches/cocotb	no	no	no	yes
testbenches/verilog	no	no	no	yes
netlist/schematic	yes	yes	yes	only AMS-facing views
netlist/layout	yes	yes	yes	not principal digital representation
netlist/pex	yes	yes	yes	yes where AMS/PEX required
netlist/nl,pnl	no	no	no	yes
netlist/xspice	no	no	no	reusable AMS digital macro
verification/drc,lvs	yes	yes	yes	via LibreLane/reports
verification/cace	yes	yes	normally child macro	no
final	integration output	integration output	submission output	integration output
macros/	not in leaf	not in leaf	yes	top-level can have it
floorplan/	no	no	HeiChips-specific	flow config instead
fpga/	no	no	no	optional/yes
parent purpose	chip macro	project submacro	tapeout submission	hardened digital macro

(raw.githubusercontent.com)

12. What I would regard as the emerging “recommended” structure

If you want something suitable for your own reusable analog/mixed-signal IP, I would take the common denominator rather than copy either repository literally:

```
my_macro/
├── Makefile
├── README.md
├── schematic/
│   └── xschem/
├── layout/
├── rtl/                                # only if applicable
├── testbenches/
│   ├── xschem/
│   │   └── plot_simulations/
│   ├── cocotb/                        # if applicable
│   └── verilog/                       # if applicable
├── flow/
│   └── librelane/                     # digital/hybrid only
├── scripts/
│   └── sizing/                       # analog where useful
├── macros/                           # only for real hierarchical IP
│   ├── <child>/
│   │   ├── Makefile
│   │   └── ...
├── netlist/
│   ├── schematic/
│   ├── layout/
│   ├── pex/
│   ├── nl/                           # digital as applicable
│   ├── pnl/
│   ├── spice/
│   └── xspice/
```

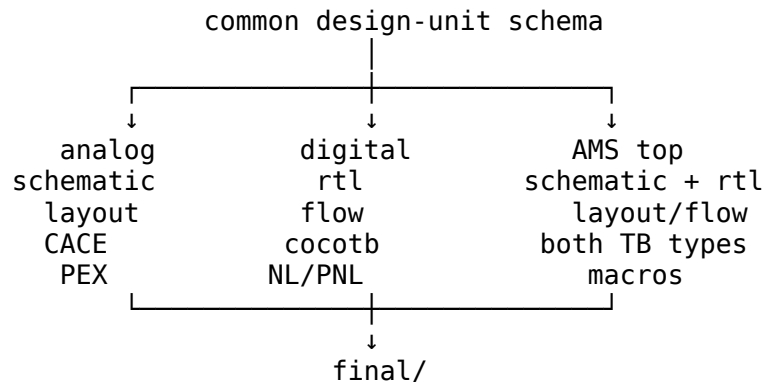
```

|
├── verification/
│   ├── drc/
│   ├── lvs/
│   ├── cace/           # analog characterization
│   └── reports/        # synthesis/STA/etc.
├── render/
│   └── img/
└── final/
    ├── gds/
    ├── lef/
    ├── lib/
    └── vh/

```

Then let the Makefile activate only the relevant subsets.

So I would **not** have separate incompatible “analog”, “digital”, and “mixed-signal” directory standards. I'd view them as profiles of one schema:



That appears to be exactly the direction the JKU and HeiChips material has converged toward. The workshop documentation even describes the macro Makefile as the single entry point for simulation, verification, building and cleanup, while the JKU full-chip documentation generalizes that same convention recursively to every macro/IP component. (heichips.github.io)

One detail I would deliberately copy from the **new JKU template rather than the HeiChips workshop copy** is the distinction between `render/img/` and `final/`: renders are documentation/debug artifacts, whereas `final/` should ideally contain only things that form the formal interface to the parent/integration flow. The JKU tree is semantically cleaner on that point. (raw.githubusercontent.com)

User

The Austrian School of repository and Makefile organization looks fairly consistent.

And Now For Something Completely Different: Mr Potbox Tim Edwards is Old School and uses Tk/Tcl and bash scripts instead of python, yaml, toml and whatnot for scripting.

How can this be consolidated into a coherent superstructure? Start looking at https://github.com/RTimothyEdwards/chipalooza_projects_1 https://github.com/RTimothyEdwards/chipalooza_projects_2 https://github.com/RTimothyEdwards/sg13cmos5l_ocr_openframe https://github.com/RTimothyEdwards/caravel_openframe_project https://github.com/RTimothyEdwards/gf180mcu_ocr_openframe https://github.com/RTimothyEdwards/gf180mcu_caravel_padframe https://github.com/RTimothyEdwards/caravel_testing and generally in Tim's repositories. Also have a look at <https://opencircuitdesign.com/>

Assistant

I'm treating this as an architecture problem, not a style war: identify Tim Edwards' recurring repository/script conventions, map them onto the JKU/HeiChips Makefile hierarchy, then propose a superstructure that lets Tcl/bash remain first-class rather than being wrapped into Python for no reason.

Assistant

A pattern is emerging that is almost the inverse of the Austrian one: JKU/HeiChips organize primarily by **design activity/lifecycle** (schematic, layout, netlist, verification, final), while Tim's older Caravel/OpenFrame lineage often organizes by **representation/tool format** (mag, gds, lef, verilog, xschem, lvs, openlane). That distinction is the key to making them coexist cleanly.

Assistant

Yes. There is a coherent superstructure here, but I would **not** try to standardize the implementation languages. The useful standardization boundary is one level higher.

The clean synthesis is:

Austrian filesystem semantics + GNU Make as public build API + Edwards-style tool-native drivers underneath.

Or, even shorter:

WHAT? → repository hierarchy
WHEN? → Make
HOW? → Tcl / shell / Python / native config

That keeps the best part of both traditions.

1. First correction: Tim is less “Tcl instead of Python” than it initially looks

His repositories are certainly Tcl/Unix-heavy, but they are not doctrinaire.

For example, sg13cmos5l_oed_openframe/scripts/ contains all of these simultaneously:

```
gen_padframe.tcl
gen_padframe_template.tcl
layout_setup.tcl
run_gen_hk_def.tcl
text.tcl

run_gen_filled_final_gds.sh
run_gen_hk_def.sh
run_gen_openframe_gds.sh
run_gen_padframe.sh
run_gen_wrapper_gds.sh
setup.sh

generate_fill.py
parse_config.py
set_user_id.py
tech_change.py
```

So the actual Edwards rule looks more like:

use the language closest to the problem.

Magic automation is Tcl because **Tcl is Magic's command language**. Netgen is similarly Tcl-based and explicitly supports batch Tcl operation. Shell connects Unix processes. Python appears where parsing or structured transformation benefits from it. (github.com)

Even his newer analog IP work incorporates CACE and therefore YAML when YAML is the appropriate declarative input format. So I would not make “no YAML/Python” part of the architecture. (github.com)

And Tk should really be separated from Tcl in our mental model. Tk is the GUI toolkit. For an automated flow, the significant component is **Tcl**; Magic can run headlessly and Netgen can run in batch mode. (opencircuitdesign.com)

2. The real difference is the first directory axis

This is more fundamental than programming language.

The Austrian hierarchy tends to answer:

What role does this artifact have in the design flow?

Hence:

```
schematic/  
layout/  
testbenches/  
netlist/  
verification/  
final/  
flow/
```

Then the tool or representation appears below it:

```
schematic/xschem/  
testbenches/xschem/  
flow/librelane/
```

Tim/Caravel-derived repositories often answer first:

What representation/tool owns this artifact?

Hence top-level directories like:

```
mag/  
gds/  
lef/  
verilog/  
xschem/  
openlane/  
lvs/  
netlist/
```

You see that repeatedly in Chipalooza, Caravel/OpenFrame, and related trees. `chipalooza_projects_1`, for example, has `gds`, `lvs`, `mag`, `netlist`, `openlane`, `verilog`, and `xschem` at the top level; Chipalooza 2 follows much the same pattern. (github.com)

Likewise the SG13 openframe tree has `def`, `gds`, `librelane`, `magic`, `netlist`, `verilog`, and `xschem`. (github.com)

That's why trying to merge the two directory structures literally becomes ugly.

The solution is to make the **semantic role the first coordinate and tool/representation the second coordinate**.

3. The superstructure I would use

Something like this:

```
my_design/  
├─ Makefile  
├─ README.md  
└─ config.mk
```

```

├── schematic/
│   ├── xschem/
│   │   ├── top.sch
│   │   └── ...
│   └── ...
├── layout/
│   ├── magic/
│   │   ├── top.mag
│   │   └── ...
│   └── gds/
│       # only if GDS itself is editable/source IP
├── rtl/
│   ├── top.sv
│   └── ...
├── testbenches/
│   ├── xschem/
│   ├── verilog/
│   └── cocotb/
├── flow/
│   ├── magic/
│   │   ├── assemble.tcl
│   │   ├── extract.tcl
│   │   ├── streamout.tcl
│   │   └── ...
│   ├── netgen/
│   │   └── lvs.tcl
│   ├── klayout/
│   ├── librelane/
│   │   └── config.*
│   └── cace/
├── scripts/
│   ├── setup.sh
│   ├── generate_fill.py
│   └── ...
└── macros/

```


layout/magic/

rather than Tim's root-level:

mag/

That says simultaneously:

role = editable layout

representation = Magic database

Likewise:

flow/magic/extract.tcl

says:

role = flow implementation

tool = Magic

instead of making .tcl or magic/ itself the semantic category.

4. Do not organize scripts/ primarily by programming language

I would avoid this:

scripts/

├─ bash/

├─ python/

└─ tcl/

because that's programmer-centric rather than design-flow-centric.

If there are enough scripts to require subdivision, this is better:

flow/

├─ magic/

├─ netgen/

├─ librelane/

├─ klayout/

└─ packaging/

scripts/

├─ dependencies/

├─ reporting/

└─ utilities/

A Magic driver happens to be Tcl. A helper that munges DEF might happen to be Python. A launch wrapper might happen to be Bash. The extension already tells you the programming language. This is one place where I think the synthesis can actually improve on **both** families of repository.

5. GNU Make should be the constitutional layer

This is where the two schools are already converging.

Tim's SG13 OpenFrame README says explicitly that the individual scripts are intended eventually to be collected behind a Makefile, with steps becoming `make <something> operations`. (github.com)

And in the Chipalooza projects Make is already largely a **facade/router/orchestrator**. It manages environment variables, obtains IP, descends recursively into other flows, invokes verification, and even forwards targets to the Caravel Makefile. (github.com)

That's exactly what I would formalize.

The user-facing API should be stable:

```
make setup
make deps
```

```
make schematic
make sim
```

```
make layout
make drc
make lvs
make pex
```

```
make cace
```

```
make build
make verify
make export
make signoff
```

```
make all
```

```
make clean
make distclean
```

with optional escape hatches:

```
make magic-drc
make klayout-drc
```

```
make netgen-lvs
make klayout-lvs
```

```
make sim-schematic
make sim-pex
```

The distinction is valuable:

```
make lvs
```

means **intent**.

```
make netgen-lvs
```

means **implementation**.

That means a project can switch verification implementation without breaking its top-level API.

6. Tcl then becomes exactly what it ought to be

Suppose Make wants a Magic extraction.

Don't do:

```
Make
↓
Python Magic wrapper
↓
subprocess
↓
Magic
↓
Tcl commands
```

unless Python genuinely adds something.

Do:

```
Make
↓
magic -dnull ... flow/magic/extract.tcl
↓
Magic/Tcl
```

Magic itself is a Tcl application environment; invoking its API in its native command language is not nostalgic eccentricity. It removes an unnecessary abstraction layer. The same argument applies to Netgen's Tcl interface. (opencircuitdesign.com)

The shell's job is different:

```
Make
↓
bash
├── set up environment
├── create directories
├── pipe/filter files
├── invoke several Unix programs
└── handle process-level glue
```

And Python has a third role:

```
Make
↓
Python
├── parse nontrivial structured data
├── transform netlists/configurations
├── generate reports
└── manipulate geometry/data models
```

That is in fact roughly what Tim's own mixed Tcl/.sh/.py script sets demonstrate. (github.com)

7. The configuration hierarchy should also have clear jurisdiction

This is where modern EDA repositories easily become an Augean stable:

```
Makefile
config.mk
.env
config.yaml
project.yaml
config.toml
settings.json
foo.tcl
bar.py
...
```

all redundantly expressing the same TOP, PDK, and paths.

I would establish this precedence:

Information	Authority
machine/site installation	environment
project-wide scalar choices	config.mk
dependencies/build graph	Make
Magic/Netgen procedure	Tcl
LibreLane configuration	LibreLane's native configuration format
CACE specification	CACE YAML
complex transformations	Python
disposable command glue	shell

Thus:

```
# config.mk
```

```
PROJECT := my_adc
TOP      := my_adc_top
PDK      := ihp-sg13g2
```

```
export PROJECT TOP PDK
```

and Magic gets those values directly:

```
set top $::env(TOP)
set pdk $::env(PDK)
```

```
load $top
...
```

That is not hypothetical; Tim's Chipalooza/OpenFrame Tcl scripts already consume environment variables such as PDK_ROOT, PDK, and IP_ROOT set by the surrounding Make environment. (github.com)

So:

Make/config.mk owns policy; Tcl receives parameters rather than secretly redefining policy.

That's a very clean interface.

8. Don't invent YAML merely to look modern

This is probably the biggest philosophical point where I would side with Mr Potbox.

This:

```
project:
  top: my_adc_top
  pdk: ihp-sg13g2
```

followed by a Python program whose purpose is essentially:

```
yaml.safe_load(...)
```

so it can eventually invoke:

```
magic ...
```

is not an architectural improvement over:

```
TOP := my_adc_top
PDK := ihp-sg13g2
```

Make already has variables, dependencies, conditional rules, incremental rebuild semantics and a CLI.

Conversely, CACE's characterization data **is naturally structured**, so:

```
verification/cace/spec.yaml
```

makes sense.

Likewise a LibreLane configuration should use whatever data representation LibreLane's supported interface expects.

So the rule should be:

Declarative serialization formats are domain interfaces, not a universal configuration religion.

Tim's own repositories actually support this conclusion: old-style Tcl/shell lives alongside Python helpers, while newer IP uses CACE YAML when CACE is part of the job. (github.com)

9. The final/ boundary needs one Edwards addition: .mag

From the Austrian structure I argued:

```
final/
├─ gds/
├─ lef/
├─ lib/
└─ vh/
```

Tim's flows reveal one omission.

For a hierarchy assembled in Magic, the useful parent-facing representation may be:

```
final/mag/
```

not merely GDS.

In the SG13 OpenFrame instructions, hardened LibreLane blocks are brought back as Magic databases specifically because the .mag representation preserves useful hierarchy/reference information for subsequent Magic-level integration, alongside netlist views for LVS and mixed-mode simulation. (github.com)

So I would generalize final/ to mean:

published views through which the parent design is permitted to consume this design unit.

Thus an analog macro might publish:

```
final/  
├─ mag/  
│   └─ opamp.mag  
├─ gds/  
│   └─ opamp.gds  
├─ lef/  
│   └─ opamp.lef  
├─ spice/  
│   └─ opamp.spice  
├─ lib/  
└─ vh/
```

No single format needs to be canonically “the final result”.

That meshes particularly well with mixed-signal hierarchical design.

10. macros/, ip/, and dependencies/ should have distinct meanings

Tim's projects use dependencies/ heavily, sometimes populated by Make targets cloning repositories; the newer SG13 OpenFrame also uses git submodules for external IP. (github.com)

I'd make the distinction explicit:

macros/

= design units **owned by this project** and physically instantiated hierarchically.

ip/

= locally controlled reusable intellectual-property units, where distinction from macros/ is useful.

dependencies/

= **external repositories/packages**, not edited as part of the project.

Then each external dependency retains its native organization. That's important.

If I import a Tim-style IP with:

```
mag/  
xschem/  
verilog/
```

I should **not reorganize its repository** merely to satisfy my superstructure.

Instead the adapter says:

```
IP_ROOT := dependencies/foo
```

```
FOO_MAG_DIR    := $(IP_ROOT)/mag  
FOO_XSCHEM_DIR := $(IP_ROOT)/xschem  
FOO_RTL_DIR    := $(IP_ROOT)/verilog
```

The repository contract applies strongly to things **we own**, not coercively to third-party IP.

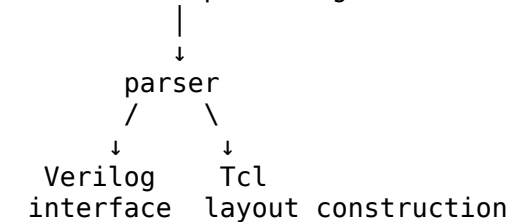
That gives us interoperability without a Great Directory Renaming Campaign.

11. This also explains Tim's config.txt

The OpenFrame project's config.txt is an interesting case. It contains project identity and pad assignment information in a simple human-readable format; parse_config.py then generates Verilog/Tcl-derived products. (github.com)

That is actually architecturally sound:

declarative pad assignment



I wouldn't replace that with YAML merely because YAML exists.

I **might** give it a more explicit name:

```
padframe.conf
```

and document its grammar.

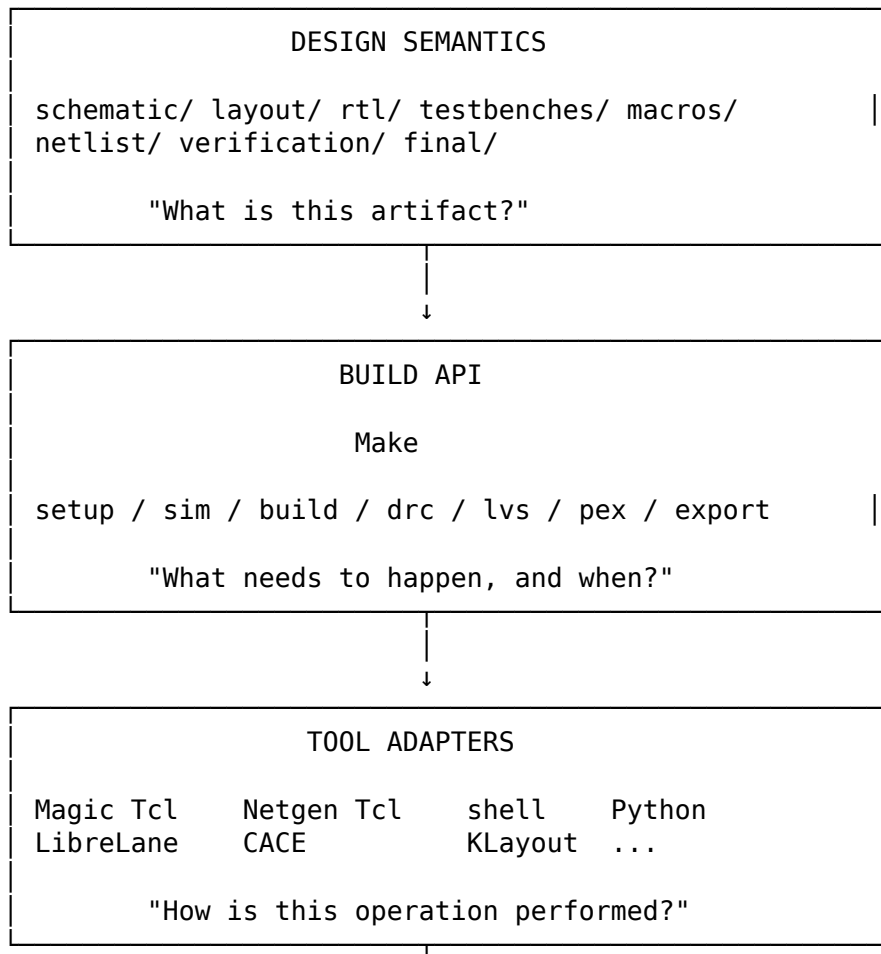
The principle is more important than serialization syntax:

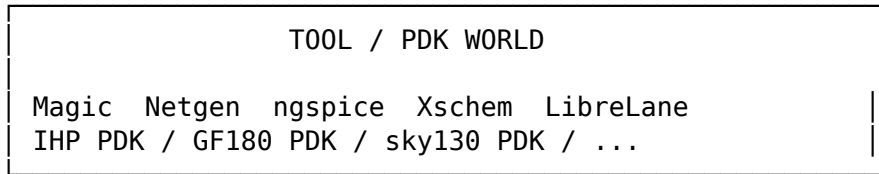
One source of truth generates all representation-specific derivatives.

That is precisely what we want in mixed-signal integration.

12. The resulting architecture has four layers

I think this is the cleanest conceptual diagram:





And configuration cuts vertically through it:

environment	→ machine/site
config.mk	→ project
CACE YAML	→ CACE
LibreLane cfg	→ LibreLane
Tcl	→ executable tool commands

13. This is actually quite close to Tim's architecture already

The SG13 OpenFrame project is particularly instructive because it is a transitional form.

It has:

```
config.txt
scripts/
magic/
xschem/
verilog/
netlist/
gds/
def/
librelane/
```

and a scripted sequence that constructs padframe, wrapper, user-project integration, stream-out, extraction and LVS. Its README explicitly anticipates exposing these operations through Make. (github.com)

The Chipalooza Makefiles already use Make as an outer control plane around external IP and subflows. (github.com)

Meanwhile Magic and Netgen make Tcl a perfectly natural inner control plane. (opencircuitdesign.com)

So we don't actually need to reconcile two incompatible philosophies.

The two traditions mostly occupy **different levels of abstraction**.

My proposed doctrine

I would call the consolidated convention:

semantic directories
+
Makefile public interface
+
tool-native flow implementation

with this one-line law:

Directories describe artifacts; Make describes dependencies; tool-native languages drive tools.

That lets the Austrian School keep its admirably legible:

schematic/
layout/
netlist/
verification/
final/

while allowing Mr Potbox to sit happily in the engine room doing:

```
load $top  
select top cell  
extract do local  
extract all
```

instead of being forced through magic-wrapper-framework-config.yaml → Python → subprocess → Tcl for the sake of modernity.

And I think the **role-first / representation-second** rule—

layout/magic/
schematic/xschem/
flow/magic/
flow/netgen/
final/gds/
final/mag/

—is the one genuinely new piece that turns the two repository cultures into a coherent system.

User

ok export verbatim as markdown and PDF, ideally that both ReText and github/gitlab can handle. github is the most important one.

Assistant

I'll export my immediately preceding answer verbatim, preserving the Markdown structure, fenced code blocks, table, and links. I'll make the Markdown GitHub/GitLab-first while keeping it plain enough for ReText, then generate a PDF from that same source.

User

Stop. Export the entire chat verbatim.